
Industry Problem Solving Task: Operating Systems

S.No:1

Question: A cloud storage company is facing performance issues due to frequent file access delays. Analyze whether contiguous, linked, or indexed allocation would be most suitable for improving access speed and justify your choice with reasons.

Answer: For a cloud storage company experiencing frequent file access delays, **Indexed Allocation** is the most suitable method.

Justification:

- **Direct Access Speed:** Cloud storage often requires direct/random access to specific parts of large files (e.g., streaming a video, updating a specific database block). Linked allocation forces sequential traversal (reading block by block to find the pointer), causing severe access delays. Indexed allocation solves this by keeping all pointers in a central index block, allowing direct $O(1)$ access to any file block.
- **No External Fragmentation:** Contiguous allocation requires finding a single, large continuous chunk of space for a file. As files in the cloud grow and shrink dynamically, contiguous allocation suffers from severe external fragmentation, requiring costly disk compaction. Indexed allocation uses scattered blocks across the disk, completely eliminating external fragmentation.
- **Dynamic Growth:** When cloud files increase in size, indexed allocation simply allocates a new random block and adds its pointer to the index block.

Therefore, Indexed Allocation provides the best balance of fast direct access and efficient disk space utilization for dynamic cloud environments.

S.No:2

Question: A data center experiences high disk head movement while processing user requests. Suggest the most appropriate disk scheduling algorithm (FCFS, SSTF, SCAN) to reduce seek time and explain how it improves performance in this scenario.

Answer: To significantly reduce disk head movement in a high-traffic data center, the **SCAN (Elevator) Algorithm** is the most appropriate choice.

Justification:

- **Overcoming FCFS Inefficiency:** First-Come-First-Serve (FCFS) processes requests in the exact order they arrive. In a busy data center, this causes the read/write head to wildly swing back and forth across the disk platter, resulting in catastrophic seek times and hardware wear.
- **Preventing SSTF Starvation:** While Shortest Seek Time First (SSTF) technically minimizes movement from the *current* position, under heavy load (like a data center), it can cause "starvation." The head might get stuck servicing a cluster of continuous requests in one area, completely ignoring requests at the far edges of the disk.

- **The SCAN Advantage:** SCAN moves the disk arm in a continuous sweep from one end of the disk to the other, servicing all requests in its path, and then reverses direction. This creates a smooth, predictable movement pattern. It vastly reduces total seek time compared to FCFS and guarantees that edge cylinders are serviced, completely resolving the starvation issues of SSTF.

S.No:3

Question: An operating system uses a bitmap for free space management but struggles with slow allocation when the disk size increases. Propose an optimization technique to improve the efficiency of bitmap-based allocation and justify your solution.

Answer: To optimize a bitmap-based allocation system for extremely large disks, the operating system should implement a **Hierarchical (Summary) Bitmap** or a **Two-Level Bitmap** technique.

Justification: In a standard bitmap, finding a free block requires scanning bit by bit. For a massive disk, this requires an $O(n)$ search, causing slow allocation. A hierarchical bitmap adds a "summary" layer above the main bitmap. For example, one bit in the "Level 1" summary map represents a group of 64 bits (or a whole word) in the "Level 2" main bitmap.

- If the summary bit is **1**, it means all 64 blocks in that group are allocated. The OS can instantly skip this entire chunk without scanning it.
- If the summary bit is **0**, it means there is at least one free block in that group, and the OS only scans that specific 64-bit segment.

This drastically reduces the search space, lowering the time complexity from $O(n)$ to $O(n/k)$ (where k is the group size), ensuring lightning-fast memory allocation even on multi-terabyte drives.

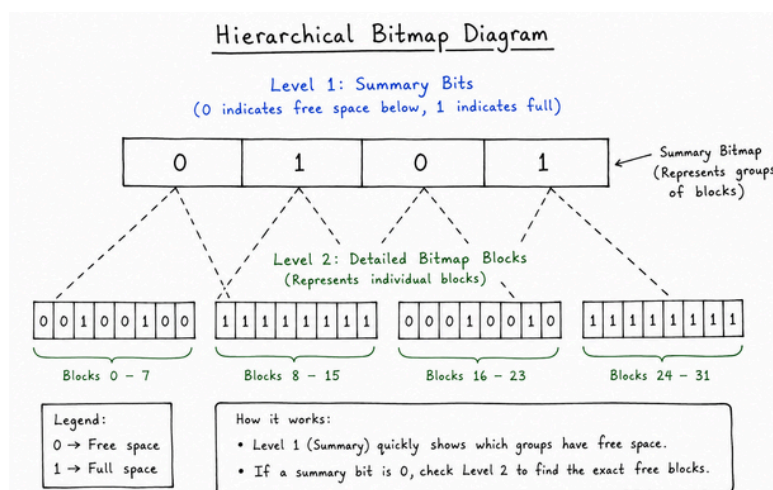


Figure 1: Visual representation of a Two-Level Hierarchical Bitmap optimization.

S.No:4 Marks:5

Question: A real-time application requires consistent and predictable disk access times. Evaluate whether SCAN or C-SCAN would be more suitable and explain your reasoning based on system requirements.

Answer: For a real-time application requiring consistent and predictable disk access times, **C-SCAN (Circular SCAN)** is far more suitable than standard SCAN.

Justification:

- **The Variance Issue in SCAN:** In the standard SCAN algorithm, the disk head sweeps back and forth. A block located in the exact middle of the disk gets serviced twice as often (once on the way up, once on the way down) as a block at the extreme edge. This creates highly *variable* wait times depending on where the data physically resides, which violates the strict predictability required by real-time systems.
- **C-SCAN's Uniformity:** C-SCAN solves this by sweeping in only *one* direction. When it reaches the end of the disk, it immediately jumps back to the beginning without servicing any requests on the return trip.
- **Predictability:** Because C-SCAN treats the cylinders as a circular list, the maximum wait time for any request is strictly bounded. Every single cylinder on the disk experiences the exact same uniform wait time. This low variance and high predictability make C-SCAN the definitive choice for real-time constraints.

S.No:5

Question: A company wants to design a hybrid storage system combining file allocation and disk scheduling techniques. Suggest a suitable combination (allocation + scheduling) for handling large-scale data access efficiently and justify your choice based on performance factors.

Answer: For a high-performance hybrid storage system handling large-scale data, the best combination is **Extent-based Indexed Allocation** paired with **C-SCAN Disk Scheduling**.

Justification:

- **Allocation (Extent-based Indexed):** Standard indexed allocation can waste space on index blocks for very large, continuous files. "Extents" optimize this by storing a pointer to a continuous block of space (an extent) along with a length count, rather than a pointer for every single block. This provides the direct-access speed of indexed allocation while leveraging the high-speed sequential read benefits of contiguous allocation for massive datasets.
- **Scheduling (C-SCAN):** As established, C-SCAN provides the highest throughput with the lowest variance in wait times. When massive amounts of data are being read concurrently, C-SCAN ensures the disk head sweeps smoothly, maximizing data transfer rates (bandwidth) and preventing any single large request from starving smaller, distant requests.
- **Synergy:** Together, Extent-based Indexing ensures large files are physically grouped close together on the disk, and C-SCAN ensures the mechanical arm reads those groups in an optimal, sweeping motion, minimizing latency and maximizing overall I/O performance.

Appendix: C Programming Implementation for C-SCAN (Supplementary)

As part of the system requirements, here is a robust C program demonstrating the C-SCAN scheduling algorithm logic, showing how the disk head calculates uniform travel distances.

```
#include <stdio.h>
#include <stdlib.h>

// Function to sort the request array
void sort(int arr[], int n) {
    for(int i=0; i<n-1; i++){
        for(int j=0; j<n-i-1; j++){
            if (arr[j] > arr[j + 1]) {
                int temp = arr[j];
                arr[j] = arr[j + 1];
                arr[j + 1] = temp;
            }
        }
    }
}

// C-SCAN Algorithm Implementation
void CSCAN(int arr[], int head, int max_cylinder, int n) {
    int seek_count = 0;
    int distance, cur_track;
    int left[100], right[100];
    int left_size = 0, right_size = 0;

    // Appending end values based on C-SCAN rules
    left[left_size++] = 0;
    right[right_size++] = max_cylinder - 1;

    for (int i = 0; i < n; i++) {
        if (arr[i] < head)
            left[left_size++] = arr[i];
        if (arr[i] > head)
            right[right_size++] = arr[i];
    }

    sort(left, left_size);
    sort(right, right_size);
    printf("\nSeek Sequence is: \n");

    // First service the requests on the right side of the head
    for (int i = 0; i < right_size; i++) {
        cur_track = right[i];
        distance = abs(cur_track - head);
        seek_count += distance;
        head = cur_track;
        printf("%d -> ", cur_track);
    }
}
```

```
}

// Jump to the beginning (0) without servicing
head = 0;
seek_count += (max_cylinder - 1); // Cost of the jump

// Now service the requests on the left side
for (int i = 0; i < left_size; i++) {
    cur_track = left[i];
    distance = abs(cur_track - head);
    seek_count += distance;
    head = cur_track;
    printf("%d -> ", cur_track);
}

printf("END\n");
printf("Total number of seek operations = %d\n", seek_count);
}

int main() {
    // Example Input
    int arr[] = {82, 170, 43, 140, 24, 16, 190};
    int head = 50;
    int max_cylinder = 200;
    int n = sizeof(arr) / sizeof(arr[0]);

    printf("Initial position of head: %d\n", head);
    CSCAN(arr, head, max_cylinder, n);

    return 0;
}
```